User, access control and tenancy (v2)

Status: specification v2, supersedes the previous tab · **Applies to:** the Tenancy and IAM modules and every protected endpoint · **Stack:** PHP 8.4, Symfony 8.1, API Platform 5, Doctrine ORM 3, PostgreSQL 17–18 · **Grounded in:** NIST RBAC (ANSI INCITS 359), ARBAC97, NIST SP 800-63B-4, OWASP ASVS 5.0 V8, PostgreSQL 18 and Symfony 8 documentation, REPROMAS profile pattern

1. Overview

A global **User** signs in once and holds one **UserProfile** per society (tenant). Roles are granted to the **profile**, never to the user, so every grant lives inside one society. Permissions are a global catalogue declared in code; roles bundle permissions; **UserRoleAssignRole** says which roles a role holder may assign. Tenants, their databases, plans and jobs live in the landlord database next to IAM.

Plane	Database	Modules	Holds
Control plane	Landlord (one)	**Tenancy**, **IAM**	Tenant registry, sign-up policy, tenant databases, domains, plans, jobs; users, profiles, credentials, tokens, API keys; permissions, roles, grants, audit
Data plane	One per society	Business modules	Units, ventures, requests, inventory, ledger and the rest (Multi-tenancy tab)

Out of scope (section 21): the approval workflow (its own module, plugging in through assertions) and cooperative membership.

2. Key decisions

Topic	Decision
Membership of a society	`user_profile (user_id, tenant_id)`, unique per pair, created automatically at sign-up and reactivated on rejoin

Topic	Decision
Platform staff	Colsynq is itself a tenant (`kind = platform`); staff hold ordinary profiles there; no nullable tenant anywhere
Roles	System roles shared (`tenant_id` null); custom roles per tenant; tenant and scope immutable
Permissions	One global catalogue keyed by slug, generated from code
Assignment authority	UserRoleAssignRole edges (`allow` / `deny`) plus a `can_assign_all` wildcard; editors may add only permissions they hold
Separation of duties	Permission-level conflicts checked on a profile's effective permissions
Tokens	JWT holds only the user ID and authentication facts; no tenant, profile or roles; access loaded per request and cached by `access_version`
Tenant resolution	Listener at `kernel.request` priority 20 (after the router, before the firewall); requirement declared per route
Record-level rules	Assertions registered by the module that owns the record
Decision strategy	Symfony `unanimous`; any deny wins
History	Rows hold current state; who, why and before/after go to one append-only `audit_event`
Integrity	Composite tenant keys and constraint triggers for every cross-row rule; each has a test

3. Modules and boundaries

```
graph TD
    subgraph Landlord [Landlord database]
        TEN[Tenancy: tenant, tenant_database, tenant_domain, plan, feature,
plan_feature, subscription, tenant_entitlement_override, tenant_job]
        IAM[IAM: app_user, user_profile, user_authenticator, backup_code,
external_identity, refresh_token, api_key, one_time_token, permission, role,
role_permission, user_role, user_role_assign_role, permission_conflict,
platform_tenant, support_access_grant, scope_reference, audit_event]
    end
    subgraph Tenants [Tenant databases]
        BIZ[Business modules]
    end
    TEN -- TenantProvisioned, TenantStatusChanged, TenantPlanChanged,
TenantDeleted --> IAM
    IAM -- UserProfileCreated, UserProfileUpdated, UserErased --> BIZ
    BIZ -- UnitActivated, UnitClosed, VentureActivated, VentureClosed --> IAM
```

Module	Schema	Owns
Tenancy	`tenancy`	Tenants and sign-up policy, tenant databases, custom domains, plans and features, subscriptions, entitlement overrides, tenant jobs, the Tenant Resolver
IAM	`iam`	Identity (users, profiles, authenticators, external identities, refresh tokens, API keys, one-time tokens) and access (permissions, roles, grants, assignment edges, conflicts, support grants, scope references, audit)
Shared kernel	none	`TenantContext` interface, `ResolvedTenant`, `#[TenantRequirement]`, `RoleScope`, `PermissionProvider`, `AccessAssertion`, `AssertionResult`, `#[ScopedBy]`

Rules:

- Tenancy and IAM exchange events only. IAM reads tenant facts (status, sign-up policy, plan features) from the shared `TenantContext`, never from Tenancy's tables.

- Inside IAM, `Iam\Identity` and `Iam\Access` are namespaces of one module: one schema, real foreign keys, one transaction for a profile and its grants.
- Business modules never import IAM. They declare permission slugs and assertions through the shared kernel and name slugs in `security` expressions.

4. Entity-relationship model

```
erDiagram
    TENANT ||--o| TENANT_DATABASE : "stored in"
    TENANT ||--o{ TENANT_DOMAIN : "custom domains"
    TENANT ||--o{ SUBSCRIPTION : "subscribes"
    PLAN ||--o{ SUBSCRIPTION : "chosen in"
    PLAN ||--o{ PLAN_FEATURE : "includes"
    FEATURE ||--o{ PLAN_FEATURE : "in"
    TENANT ||--o{ TENANT_ENTITLEMENT_OVERRIDE : "overrides"
    TENANT ||--o{ TENANT_JOB : "runs"
    APP_USER ||--o{ USER_PROFILE : "one per tenant"
    APP_USER ||--o{ USER_AUTHENTICATOR : "proves identity"
    USER_AUTHENTICATOR ||--o{ BACKUP_CODE : "holds"
    APP_USER ||--o{ EXTERNAL_IDENTITY : "federates"
    APP_USER ||--o{ REFRESH_TOKEN : "sessions"
    APP_USER ||--o{ ONE_TIME_TOKEN : "links"
    USER_PROFILE ||--o{ API_KEY : "service accounts"
    USER_PROFILE ||--o{ USER_ROLE : "holds"
    ROLE ||--o{ USER_ROLE : "granted as"
    SCOPE_REFERENCE ||--o{ USER_ROLE : "scopes"
    ROLE ||--o{ ROLE_PERMISSION : "includes"
    PERMISSION ||--o{ ROLE_PERMISSION : "in"
    ROLE ||--o{ USER_ROLE_ASSIGN_ROLE : "may assign"
    PERMISSION ||--o{ PERMISSION_CONFLICT : "conflicts with"
    USER_PROFILE ||--o{ SUPPORT_ACCESS_GRANT : "grants or uses"
```

27 tables: 9 in `tenancy`, 18 in `iam`. Foreign keys are real inside a schema; `tenant_id` links from `iam` to `tenancy` are logical UUIDs.

For a plain-language explanation of every table and field (what it holds, why it exists, how it is set and used), see section 25.

5. Database and mapping conventions

Topic	Standard
Names	`snake_case`, singular tables, one schema per module; constraints `<table>_<columns>_<pk, fk, uq, ck or idx>`
Primary keys	UUID v7 generated in the application; `DEFAULT uuidv7()` once on PostgreSQL 18
Time	`timestamptz`; UTC in the application
Case-insensitive values	Normalised by one shared normaliser in PHP and SQL (email: trim, NFC, lower-case; slug: lower-case); plain unique index; no `citext`
Value sets	`text` + `CHECK (col IN (…))`, mapped to PHP backed enums; no native `ENUM` types
Nullable tenant (system rows)	A pair of partial unique indexes (`WHERE tenant_id IS NULL` and `WHERE tenant_id IS NOT NULL`), mapped in Doctrine with `options: {where: …}`
"One active row"	Partial unique indexes, never table constraints (a constraint cannot have `WHERE`)
Composite tenant keys	`(child_id, tenant_id)` → `parent(id, tenant_id)` keeps a child in its parent's tenant. Doctrine maps the association on `id` plus a plain `tenant_id` column; the composite FK lives in a hand-written migration and is checked by a test
Rules a foreign key cannot express	`CONSTRAINT TRIGGER … DEFERRABLE INITIALLY IMMEDIATE` named `<table>_<rule>_ck`, mirrored by an application check and a test
Foreign keys	Every referencing column indexed
jsonb	Variable or opaque data only (`metadata`, `public_data`, audit payloads)
Secrets	Hashes or encrypted `bytea`; vault references for database credentials

Topic	Standard
Concurrency	Counters bumped with `x = x + 1`; cross-row invariants under `pg_advisory_xact_lock`
Audit	Append-only, monthly partitions, primary key `(occurred_at, id)`; application role has INSERT and SELECT only; a maintenance role manages partitions
Retention and erasure	Audit kept 7 years, then partitions dropped; payloads hold IDs, not emails or phones; erasure blanks personal columns and keeps IDs (NDPA 2023)

6. Tenancy tables (`tenancy` schema)

6.1 tenant

Column	Type	Constraints and notes
`id`	uuid	PK, v7
`kind`	text	`platform` or `society`; partial unique index: exactly one `platform` row
`slug`	text	Unique; `^[a-z0-9][a-z0-9-]{1,62}$`; sent in `X-Tenant`, used as subdomain
`legal_name`, `short_name`	text	
`registration_number`, `registering_authority`, `state`	text	Cooperative registration; null for the platform tenant
`status`	text	`provisioning`, `active`, `suspended`, `archived`, `deleting`, `deleted`, `failed`
`status_reason`	text	Required for `suspended`, `archived`, `failed` (CHECK)

Column	Type	Constraints and notes
`status_changed_at`	timestamptz	
`activated_at`	timestamptz	First activation
`signup_mode`	text	`open`, `email_domain`, `closed` (default)
`signup_email_domains`	text[]	Non-empty only when `signup_mode = email_domain` (CHECK)
`require_email_verification`	boolean	Default true; CHECK `signup_mode <> 'email_domain' OR require_email_verification`
`region`, `timezone`, `currency`	text	`ng`, `Africa/Lagos`, `NGN`
`logo_url`, `primary_colour`	text	Public branding
`created_by_profile_id`	uuid	A platform-tenant profile (logical link)
`created_at`, `updated_at`	timestamptz	

The sign-up role is not configurable: self sign-ups and joins always receive the system role `SELF_SERVICE`.

6.2 tenant_database (one per society tenant)

A separate 1:0..1 table because it holds credential references under a stricter grant, exists only for society tenants, and is written by jobs far more often than `tenant`.

Column	Type	Constraints and notes
`tenant_id`	uuid	PK, FK `tenant`
`db_host`, `db_port`	text, integer	Database server
`db_name`	text	Unique; generated
`db_role`	text	Per-tenant runtime role
`secret_ref`	text	Vault path; never the password

Column	Type	Constraints and notes
`schema_version`	text	Cache of the tenant database's `doctrine_migration_versions`
`schema_status`	text	`ok`, `migrating`, `failed`, `drift`
`migration_wave`	smallint	0 canary to 3 last
`provisioned_at`	timestamptz	

6.3 Other tenancy tables

Table	Columns	Constraints and notes
`tenant_domain`	`host` PK, `tenant_id` FK, `is_primary`, `verification_token_hash`, `verified_at`	Custom domains only (subdomains derive from the slug); partial unique `(tenant_id) WHERE is_primary`
`feature`	`code` PK, `name`, `description`	Plan features
`plan`	`id` PK, `code`, `version`, `name`, `is_active`	Unique `(code, version)`
`plan_feature`	`plan_id` FK, `feature_code` FK, `limit_value`	PK `(plan_id, feature_code)`; null limit = unlimited
`subscription`	`id`, `tenant_id` FK, `plan_id` FK, `status` (`trialing`, `active`, `past_due`, `cancelled`, `expired`), `starts_at`, `ends_at`	Partial unique `(tenant_id) WHERE status IN ('trialing','active','past_due')`
`tenant_entitlement_override`	`tenant_id` FK, `feature_code` FK, `is_enabled`, `limit_value`, `expires_at`, `reason`	PK `(tenant_id, feature_code)`

Table	Columns	Constraints and notes
`tenant_job`	`id`, `tenant_id` FK, `kind` (`provision`, `migrate`, `move`, `archive`, `delete`), `rollout_id`, `step`, `status` (`queued`, `running`, `succeeded`, `failed`), `attempt`, `from_version`, `to_version`, `details` jsonb, `error`, `started_at`, `finished_at`	Partial unique `(tenant_id) WHERE status = 'running'` (per-tenant lock); index `(rollout_id)`

7. IAM tables: identity

7.1 app_user (entity `User`)

Column	Type	Constraints and notes
`id`	uuid	PK, v7; JWT `sub`
`kind`	text	`person` or `service`
`email`	text	Normalised; unique where not null; required for `person` (CHECK)
`phone`	text	E.164; unique only where `phone_verified_at` is not null; never the only recovery factor
`password_hash`	text	Argon2id; null for service users and passkey-only users
`status`	text	`active` or `deactivated`
`token_version`	integer	JWT `ver`; bumped to sign out everywhere
`failed_login_count`	integer	Reset on success
`locked_until`	timestamptz	Short, growing back-off (minutes) so attackers cannot lock users out

Column	Type	Constraints and notes
`email_verified_at`, `phone_verified_at`	timestamptz	Null = unverified
`trusted_device_version`	integer	Bumped to forget trusted devices
`last_login_at`, `created_at`, `updated_at`	timestamptz	

Derived, not stored: MFA enabled (an active authenticator), platform staff (an active platform-tenant profile), locked, verified.

7.2 user_profile (entity `UserProfile`)

Column	Type	Constraints and notes
`id`	uuid	PK, v7
`user_id`	uuid	FK `app_user`, NOT NULL
`tenant_id`	uuid	NOT NULL
`first_name`, `last_name`	text	As known in this society
`status`	text	`active`, `disabled`, `removed`
`status_reason`, `status_changed_at`	text, timestamptz	
`source`	text	`self_signup`, `admin_created`, `tenant_bootstrap`, `join_link` (first entry; rejoins are audited)
`access_version`	integer	Bumped in the same transaction as any change to the profile's grants, status or its roles' permissions
`created_by_profile_id`	uuid	FK `(created_by_profile_id, tenant_id)` → `user_profile(id, tenant_id)`, or a platform-staff profile for bootstrap (trigger); null for self sign-up

Column	Type	Constraints and notes
`metadata`	jsonb	Any extra information a society collects; never used for sign-in, access or uniqueness
`created_at`, `updated_at`	timestamptz	

Constraints: unique `(user_id, tenant_id)`; unique `(id, tenant_id)` (target of composite keys); index `(tenant_id, status)`. A service user has exactly one profile (trigger `user_profile_service_single_ck`).

7.3 Credentials and sessions

Table	Columns	Constraints and notes
`user_authenticator`	`id`, `user_id` FK, `type` (`totp`, `email_otp`, `passkey`, `backup_codes`), `label`, `credential_id` bytea, `public_data` jsonb, `secret_encrypted` bytea, `confirmed_at`, `last_used_at`, `revoked_at`, `created_at`	Unique `(credential_id)` where not null; index `(user_id) WHERE revoked_at IS NULL`; none for service users (trigger)
`backup_code`	`authenticator_id` FK, `code_hash`, `used_at`	One row per code
`external_identity`	`id`, `user_id` FK, `provider`, `subject`, `email`, `email_verified`, `linked_at`, `last_used_at`, `unlinked_at`	Unique `(provider, subject)`
`refresh_token`	gesdinet 3 entity: token hash, `username` (= user ID), `valid`, `family`, `family_valid`, `origin`	`hash_tokens` enabled (opt-in), `single_use` and reuse detection on; `origin` binds a token family to the issuing society host

Table	Columns	Constraints and notes
`api_key`	`id`, `user_profile_id`, `tenant_id`, `name`, `prefix`, `lookup_hash`, `ip_allow_list` cidr[], `expires_at`, `last_used_at`, `revoked_at`, `created_at`	FK `(user_profile_id, tenant_id)` → `user_profile(id, tenant_id)`; unique `lookup_hash` = HMAC-SHA-256 with a server-side pepper; trigger `api_key_service_profile_ck`
`one_time_token`	`id`, `purpose` (`signup`, `password_reset`, `set_password`), `selector`, `verifier_hash`, `user_id` (null for `signup`), `tenant_id` (for `signup`), `payload_encrypted` bytea, `expires_at`, `used_at`, `created_at`	Unique `selector`; `signup` rows hold a pending registration until the email is verified

Not stored: email-verification and join links (signed URLs), the JWT blocklist and refresh-reuse cache (Redis).

8. IAM tables: access

8.1 permission (catalogue)

Column	Type	Constraints and notes
`slug`	text	**PK**; `resource:action`; no wildcards
`resource`, `action`	text	CHECK: slug equals resource, colon, action
`module`	text	Owning module
`name`, `description`	text	For the role editor
`risk`	text	`low`, `high`, `money`
`allowed_scopes`	text[]	Subset of role scopes; platform-only permissions have `{PLATFORM}`

Column	Type	Constraints and notes
`feature_code`	text	Plan feature, nullable (logical link)
`deprecated_at`	timestamptz	Kept one release, then deleted with its role and conflict rows (ON DELETE CASCADE)

Written only by `iam:catalogue:sync` from code.

8.2 role (catalogue)

Column	Type	Constraints and notes
`id`	uuid	PK, v7
`tenant_id`	uuid	Null = system role; otherwise a custom role of that tenant; immutable (trigger)
`code`	text	Partial unique `(code) WHERE tenant_id IS NULL` and `(tenant_id, code) WHERE tenant_id IS NOT NULL`; a custom code may not equal a system code (trigger)
`name`, `description`	text	
`scope`	text	`PLATFORM`, `TENANT`, `BUSINESS_UNIT`, `VENTURE`, `SELF`; immutable; CHECK `scope <> 'PLATFORM' OR tenant_id IS NULL`
`can_assign_all`	boolean	Assignment wildcard within the tenant
`based_on_role_id`	uuid	FK `role`; a system role or the same tenant's (trigger)
`max_grant_duration`	interval	When set, every grant needs `expires_at` within it
`archived_at`	timestamptz	Archiving revokes active grants; archived roles cannot be granted; "delete" means archive
`created_at`, `updated_at`	timestamptz	

8.3 role_permission

Column	Type	Constraints and notes
`role_id`	uuid	FK `role` ON DELETE CASCADE
`permission_slug`	text	FK `permission` ON DELETE CASCADE
`created_at`	timestamptz	

PK `(role_id, permission_slug)`; index `(permission_slug)`. Rules (trigger plus application): the permission's `allowed_scopes` contains the role's scope; an editor may add only permissions they hold in the tenant; `money` and `high` permissions need `role-permissions:manage-privileged`, which no system role has by default; system roles change only through the catalogue release; every change bumps holders' `access_version` and re-checks conflicts.

8.4 user_role (grant)

Column	Type	Constraints and notes
`id`	uuid	PK, v7
`user_profile_id`	uuid	NOT NULL
`tenant_id`	uuid	NOT NULL; FK `(user_profile_id, tenant_id)` → `user_profile(id, tenant_id)`
`role_id`	uuid	FK `role`; trigger `user_role_role_tenant_ck`: system role or this tenant's, not archived; platform roles only in the platform tenant, and only platform roles there
`scope_ref_id`	uuid	Unit or venture ID for `BUSINESS_UNIT` and `VENTURE` roles; null otherwise; FK `(tenant_id, scope_ref_id)` → `scope_reference`; kind matches the role's scope (trigger)
`granted_at`	timestamptz	
`expires_at`	timestamptz	Required within `max_grant_duration`; not allowed on `TENANT_ADMIN` and `PLATFORM_OWNER`

Column	Type	Constraints and notes
`revoked_at`	timestamptz	Rows never deleted; who and why in `audit_event`

Indexes: unique `(user_profile_id, role_id) WHERE revoked_at IS NULL AND scope_ref_id IS NULL`; unique `(user_profile_id, role_id, scope_ref_id) WHERE revoked_at IS NULL AND scope_ref_id IS NOT NULL`; `(tenant_id, role_id) WHERE revoked_at IS NULL`; `(expires_at) WHERE revoked_at IS NULL AND expires_at IS NOT NULL`; `(role_id)`. The job `iam:grants:expire` (every minute) revokes expired grants, bumps `access_version` and audits.

8.5 user_role_assign_role (who may assign which role)

Column	Type	Constraints and notes
`id`	uuid	PK, v7
`tenant_id`	uuid	Null = default edge; set = that tenant's edge
`assigner_role_id`	uuid	FK `role` (A)
`assignable_role_id`	uuid	FK `role` (B); A may equal B (deputies)
`effect`	text	`allow` or `deny`
`created_at`	timestamptz	

Partial unique indexes for default and tenant rows; both roles are system roles or this tenant's (trigger). **Precedence:** tenant `deny` → `can_assign_all` → tenant `allow` → default `allow` → refused.

8.6 Other access tables

Table	Columns	Constraints and notes
`permission_conflict`	`id`, `tenant_id` (null = default), `permission_a`, `permission_b`, `same_scope_only`, `severity` (`block`, `warn`)	FKs to `permission` ON DELETE CASCADE; CHECK `permission_a < permission_b`; partial unique indexes; checked on effective permissions; a tenant row may only make a default stricter
`platform_tenant`	`singleton` boolean PK (always true), `tenant_id`	Seeded at bootstrap; tells triggers which tenant is the platform
`support_access_grant`	`id`, `tenant_id`, `support_profile_id`, `granted_by_profile_id`, `mode` (`read_only`, `impersonate`), `reason`, `ticket_ref`, `starts_at`, `expires_at`, `revoked_at`	FK `(granted_by_profile_id, tenant_id)` → `user_profile(id, tenant_id)`; support profile in the platform tenant (trigger); CHECK `expires_at > starts_at AND expires_at <= starts_at + interval '4 hours'`
`scope_reference`	`tenant_id`, `scope_ref_id`, `kind` (`business_unit`, `venture`), `name`, `status`, `source_version`, `updated_at`	PK `(tenant_id, scope_ref_id)`; event-fed, applied only when `source_version` is newer; closing revokes scoped grants
`audit_event`	`occurred_at`, `id`, `tenant_id`, `actor_user_id`, `actor_profile_id`, `actor_kind` (`user`, `system`, `support`), `action`, `subject_type`, `subject_id`, `payload` jsonb, `ip`, `user_agent`, `correlation_id`	PK `(occurred_at, id)`; monthly range partitions; no FKs; INSERT and SELECT only for the app; indexes `(tenant_id, occurred_at)`, `(subject_type, subject_id)`; written by DBAL

9. Role scopes

Scope	Rank	`scope_ref_id`	Applies to	Example roles
`PLATFORM`	−1	null	Colsynq; only in the platform tenant	Platform Owner, Platform Admin, Support Agent
`TENANT`	0	null	The whole society	Tenant Administrator, Finance Officer, Accountant, Auditor, Viewer
`BUSINESS_UNIT`	1	unit ID	One business unit	Unit Manager, Storekeeper, Cashier, Procurement Officer
`VENTURE`	1	venture ID	One venture	Venture Manager, Venture Accountant
`SELF`	2	null	Records owned by the profile's user	Self-service User

Grants combine by union, per permission. `RoleScope` is defined once, in the shared kernel.

10. Permission catalogue

Slugs are `resource:action`; actions are `list`, `read`, `create`, `update`, `delete`, `execute`, `export`, `manage`. Risk is `low`, `high` (personal data, configuration, access) or `money`. Approval permissions will come from the approval module.

```
#[AutoconfigureTag('colsynq.permission_provider')]
final class ProcurementPermissions implements PermissionProvider   // example
module
{
    public function permissions(): array
    {
        return [
            PermissionDefinition::of('purchase-orders', ['list', 'read',
'create', 'update'], risk: 'low',
                scopes: [RoleScope::Tenant, RoleScope::BusinessUnit],
feature: 'procurement'),
            PermissionDefinition::of('supplier-invoices', ['execute'], risk:
'money',
                scopes: [RoleScope::Tenant], feature: 'procurement'),
        ];
    }
}
```

`iam:catalogue:sync` runs on every deploy in one transaction: upsert `permission` rows, mark missing slugs deprecated, apply `SystemRoleCatalogue` (system roles, their permissions, default edges, default conflicts), delete slugs deprecated in the previous release, write `audit_event`. The catalogue's content hash is a build parameter (`catalogue_version`) used in the access cache key. Seeds that depend on ordering sort by slug.

11. System roles and defaults

Role code	Scope	Main permissions	Notes
`TENANT_ADMIN`	TENANT	Profiles, roles, low-risk role permissions, grants, assignment edges, tenant conflicts, tenant settings and sign-up policy, support grants; read of all modules	`can_assign_all`; no `money` or privileged permissions

Role code	Scope	Main permissions	Notes
`FINANCE_OFFICER`	TENANT	`money` execute permissions; read ledger and reconciliation	
`ACCOUNTANT`	TENANT	Journals, statements, reconciliation; read finance	
`AUDITOR`	TENANT	Read and export everything, including `audit_event`	No create, update, delete or execute
`EXTERNAL_AUDITOR`	TENANT	Read and export finance	`max_grant_duration` 90 days
`VIEWER`	TENANT	Read non-sensitive resources	
`UNIT_MANAGER`	BUSINESS_UNIT	Unit operations; assigns unit roles in own unit	
`STOREKEEPER`	BUSINESS_UNIT	Goods received, stock movements and counts, deliveries	
`CASHIER`	BUSINESS_UNIT	Sales, refunds, cash-ups, petty cash	
`PROCUREMENT_OFFICER`	BUSINESS_UNIT	Suppliers, quotations, purchase orders	

Role code	Scope	Main permissions	Notes
`VENTURE_MANAGER`	VENTURE	Venture operations and KPIs; assigns venture roles in own venture	
`VENTURE_ACCOUNTANT`	VENTURE	Venture journals and returns	
`SELF_SERVICE`	SELF	`me-*`; own records where a module offers self-service	Granted only by the system path
`PLATFORM_OWNER`	PLATFORM	All platform permissions	At least two holders
`PLATFORM_ADMIN`	PLATFORM	All `tenants:` and `plans:` permissions; platform profiles	
`SUPPORT_AGENT`	PLATFORM	Platform read; tenant data only under a support grant	
`COMPLIANCE_OFFICER`	PLATFORM	Tenant registration details, platform audit	
`FINANCE_OPERATIONS`	PLATFORM	Plans, subscriptions, billing	

Default assignment edges (`allow`): `UNIT_MANAGER` → `UNIT_MANAGER`, `STOREKEEPER`, `CASHIER`, `PROCUREMENT_OFFICER`; `VENTURE_MANAGER` → `VENTURE_MANAGER`, `VENTURE_ACCOUNTANT`; `PLATFORM_OWNER` → every platform role; `PLATFORM_ADMIN` → `SUPPORT_AGENT`, `COMPLIANCE_OFFICER`, `FINANCE_OPERATIONS`.

Default permission conflicts (examples; the catalogue holds the full list):

Permission A	Permission B	Same scope only	Severity	Why
`audit-events:export`	`payments:execute`	no	block	Auditors never move money
`bank-statements:manage`	`payments:execute`	no	block	Whoever pays does not reconcile
`journals:create`	`sales:create`	yes	block	A cashier does not post their own takings
`goods-received:create`	`purchase-orders:create`	yes	block	Whoever orders does not receive
`payments:execute`	`user-roles:create`	no	warn	Administrators should not also pay

Tenants create custom roles (for example President, Treasurer), optionally copying a system role. Conflicts apply to permissions, so copies are covered.

12. Tenancy lifecycle and the Tenant Resolver

12.1 Creating a society tenant

```
sequenceDiagram
    autonumber
    participant PA as Platform Admin (platform tenant profile)
    participant TEN as Tenancy
    participant JOB as tenant_job workers
    participant IAM as IAM
    participant A as First administrators
    PA->>TEN: POST /api/platform/tenants (details, plan, sign-up policy, two
admin emails)
    TEN->>TEN: tenant (provisioning), tenant_database, subscription,
tenant_job (provision)
    TEN->>JOB: role + database from template, migrate, seed
    JOB->>TEN: tenant active
    TEN-->>IAM: TenantProvisionedV1 (tenant ID, invitation reference)
    IAM->>IAM: system path: app_user (new or existing) + user_profile +
TENANT_ADMIN, one transaction
    IAM-->>A: set-password link or notice
```

Administrator emails travel in a short-lived encrypted invitation record referenced by the event, so events hold no personal data.

Status	Sign-in	Reads	Writes	Money actions
`provisioning`, `failed`	No (503)	No	No	No
`active`	Yes	Yes	Yes	Yes
`suspended`	Yes	Yes	Only paying the subscription	No
`archived`	`TENANT_ADMIN` and `AUDITOR` holders	Read-only	No	No
`deleting`, `deleted`	No (404)	No	No	No

Deleting a tenant publishes `TenantDeletedV1`; IAM removes its profiles, grants, keys and scope references, and deactivates users left with no profile.

12.2 Tenant Resolver Service

Based on REPROMAS (`TenantResolveListener`, `CurrentTenantProvider`, stamp and context middleware, `TenantScopedRunner`, tenant lookup endpoint), corrected for Symfony's listener order.

- **Route requirement:** `#[TenantRequirement(Required | Optional | None | Platform)]`, default `Required`. A route loader compiles it into the route default `_tenant`; there is no path bypass list.
- **Listener:** `kernel.request` **priority 20**, after the router (32), before the firewall (8). Unknown paths get 404 before any tenant lookup.
- **Sources, in order:** `X-Tenant` header (slug), then host (verified custom domain, then `<slug>.colsynq.app`). Header and host naming different tenants → `400 tenant_mismatch`.
- **Platform routes** resolve to the platform tenant and refuse any society tenant (404). Platform permissions only count there.
- **Result:** an immutable `ResolvedTenant` (id, kind, slug, status, sign-up policy, plan features) in `TenantContext`; reset between requests and messages.
- **Errors:** `400 tenant_required`, `400 tenant_mismatch`, `404 tenant_not_found`, `503 tenant_unavailable`.
- **Workers and webhooks:** `TenantStampMiddleware`, `TenantContextMiddleware` (reset in `finally`), `TenantScopedRunner`.
- **Public lookup:** `GET /api/tenants/resolve?slug=` returns name, logo, colours, `signup_mode` and `available: true/false`.

```
#[AsEventListener(event: KernelEvents::REQUEST, priority: 20)]   // after
RouterListener (32), before Firewall (8)
final readonly class TenantResolveListener
{
    public function __construct(private TenantResolver $resolver, private
TenantContext $context) {}

    public function __invoke(RequestEvent $event): void
    {
        $request = $event->getRequest();
        if (!$event->isMainRequest() || $request->isMethod('OPTIONS')) {
```

```
            return;
        }
        $requirement = TenantRequirement::from($request->attributes-
>get('_tenant', 'required'));
        if (TenantRequirement::None === $requirement) {
            return;
        }
        $tenant = TenantRequirement::Platform === $requirement
            ? $this->resolver->platform($request)       // 404 if a society
tenant is named
            : $this->resolver->resolve($request);       // header, then host;
cached; may throw TenantMismatch
        if (null === $tenant) {
            if (TenantRequirement::Required === $requirement) {
                throw new TenantRequired();
            }
            return;
        }
        $tenant->assertReachable();                      // 503
provisioning/failed, 404 deleting/deleted
        $this->context->switchTo($tenant);               // refuses a second
tenant; switches the tenant database
    }
}
```

13. Sign-up and profile lifecycle

Every way a User enters a tenant goes through **`RegisterUserInTenant`**, whose tenant comes from `TenantContext`, never the request body. It is idempotent per (user, tenant), under `SELECT … FOR UPDATE` or `INSERT … ON CONFLICT (user_id, tenant_id) DO NOTHING`:

Existing profile	Result
None	Create the profile; grant `SELF_SERVICE` (or the bootstrap or initial role)
`removed`	Reactivate: status `active`, audit `user_profile.rejoined`, old grants stay revoked, fresh `SELF_SERVICE`

Existing profile	Result
`disabled`	Refuse; only an administrator can enable; neutral response
`active`	No change; same response

13.1 Entry paths

Path	Endpoint	Result
Self sign-up	`POST /api/auth/register` (tenant required)	`open`, or `email_domain` with a matching domain. Nothing is created yet: a pending registration (`one_time_token` purpose `signup`) is stored and a verification link sent; opening it creates the `app_user` (verified), the profile and `SELF_SERVICE` together. An existing email gets a join link instead. Always the same 202
Join with an existing account	`POST /api/tenants/{slug}/join` (join-link token) or `POST /api/me/profiles` on an `open` tenant	Needs a verified email (and a domain match for `email_domain`); profile `join_link` + `SELF_SERVICE`
Created by an administrator	`POST /api/profiles`	New email: `app_user` without password + profile + optional initial role through the gates + set-password link (which verifies the email). Existing email: profile + notice
First tenant administrators	Section 12.1	Profile `tenant_bootstrap` + `TENANT_ADMIN` via the system path

Path	Endpoint	Result
Service account	`POST /api/service-accounts`	`app_user` (`kind = service`) + its only profile + first `api_key`
Platform staff	`POST /api/platform/profiles`	Profile in the platform tenant + platform role through the gates

Abuse protection: sign-up, join and resend are rate-limited per IP and per email; creation endpoints accept `Idempotency-Key`; emails are sent asynchronously so timing does not reveal whether an account exists.

13.2 Signing in at a tenant

`ProfileLoginValidator` runs on `CheckPassportEvent` after the password check and returns the same "invalid email or password" error and timing when there is no active profile in the resolved tenant. Without a tenant, sign-in ends at the society switcher (`GET /api/me/profiles`). The response body adds profile and access for the UI; the JWT stays tenant-free but carries `amr` and `auth_time` for per-tenant MFA policy.

13.3 Profile changes

Change	Effect	Tokens
Disable	`disabled`; grants kept but ignored; `access_version` bumped	Unchanged; that tenant returns 404
Enable	`active`; `access_version` bumped	Unchanged
Remove (or the user leaves)	`removed`; active grants revoked; a service account's keys revoked	Unchanged for persons
Deactivate the User	`app_user.status = deactivated`; `token_version` bumped; all grants revoked	Refresh tokens revoked
Erase the User (NDPA request)	Deactivate, then blank email, phone, names and metadata; IDs kept	All revoked

Disable, remove, deactivate and erase run the two-administrator rule (section 15.3) in the same transaction.

14. Request pipeline and authorisation decision

```
graph TD
    REQ[Bearer token or ApiKey + society host or X-Tenant] --> RT[Router, 32:
match route and _tenant]
    RT --> TR[1. Tenant Resolver, 20]
    TR --> FW[2. Firewall, 8: JWT or API key, load app_user]
    FW --> PR{3. Active user_profile in this tenant?}
    PR -->|yes| AC[4. EffectiveAccess]
    PR -->|no, valid support grant| SA[3b. Support access]
    PR -->|no| E404[404]
    SA --> AC
    AC --> SF[5. Scope filters on tenant entity manager]
    SF --> V[6. AccessVoter -> AccessDecision]
    V --> H[Handler, or 403 / 401 with reason]
```

- **Step 2:** an API key must belong to the resolved tenant (`400 tenant_mismatch` otherwise).
- **Step 3b:** a platform-tenant profile with an unexpired, unrevoked `support_access_grant` for this tenant gets a synthetic access set: read-only, or the impersonation limits of the Authentication tab.
- **Step 4:** read `access_version` first, then grants and permissions in the same snapshot on the primary; cache under `access:{tenant}:{profile}:{access_version}:{catalogue_version}` with Symfony Cache `get()` (stampede protection); time-to-live is 5 minutes or the earliest grant or support-grant expiry, whichever is sooner.
- **Step 5:** `business_unit_scope` , `venture_scope` and `self_scope` filters on entities marked `#[ScopedBy]` , bypassed when the profile holds the list permission at TENANT scope.

Step 6, one job per class:

Class	Job
`AccessVoter` (Infrastructure)	Thin Symfony adapter for permission-slug attributes; delegates

Class	Job
`AccessDecision` (Application)	Runs the checks in order; returns grant, deny (reason) or challenge
`PermissionCheck`	The profile holds the slug
`ScopeCheck`	A grant's scope covers the subject
`AssertionRegistry`	Runs every assertion registered for the slug

Security configuration: `access_decision_manager: { strategy: unanimous, allow_if_all_abstain: false }`; no other voter may support permission slugs (architecture test). Deny reasons go through `Vote::addReason()` into the 403.

Assertions decide whether a permitted action is allowed for this record, in this state, now. The owning module registers them (`AccessAssertion`, tag `colsynq.access_assertion`); they are read-only, fast and deterministic and return `Grant`, `Deny(reason)`, `Abstain` or `Challenge(reason)` (401).

Assertion	Owner	Rule
`TenantWritableAssertion`	Tenancy	No writes when the tenant is suspended or archived
`ArchivedTenantAccessAssertion`	Tenancy	In archived tenants only `TENANT_ADMIN` and `AUDITOR` holders read
`PlanFeatureAssertion`	Tenancy	Permission's `feature_code` must be in the plan or an override
`OwnRecordAssertion`	IAM	`SELF` grants cover only the user's own records
`StepUpAssertion`	IAM	`money` and `high` permissions, and MFA-required tenants, need MFA (`amr`) and recent authentication (`auth_time`); otherwise `Challenge` → 401 `step_up_required`
`DraftOnlyAssertion` (example)	Procurement	Purchase orders change only while draft

Assertion	Owner	Rule
(later) approval state	Approval module	Deny until the workflow completes

15. Granting and revoking roles

15.1 User grants: five gates (`UserRoleAssignmentPolicy`)

0. **Own profile:** nobody grants or revokes their own roles.
1. **Authority:** precedence in 8.5 (tenant `deny` , `can_assign_all` , tenant `allow` , default `allow`); platform roles only by platform roles in the platform tenant.
2. **Scope containment:** unit callers grant only with their own unit IDs, venture callers only their venture; `TENANT` roles only by `can_assign_all` holders; the reference must be active in `scope_reference` .
3. **Target and role:** active profile in the caller's tenant (composite FK); system role or the tenant's own, not archived (trigger); `expires_at` within `max_grant_duration` .
4. **Conflicts:** the profile's effective permissions after the grant contain no `block` pair (tenant rows first, then default); `warn` needs a reason in `audit_event` .

Revoking uses gates 0–2 with the same scope check, sets `revoked_at` , bumps `access_version` , audits and publishes `UserRoleRevokedV1` . Re-granting over an expired grant revokes and replaces it in one transaction.

15.2 System grants

The system path grants without a human caller, only for `SELF_SERVICE` (profile created or reactivated) and bootstrap `TENANT_ADMIN` . It skips gates 0–2, runs gates 3 and 4, and is audited with `actor_kind = system` . No other code may grant without a caller.

15.3 Two-administrator rule

Each society keeps at least two active `TENANT_ADMIN` holders; the platform tenant two `PLATFORM_OWNER` holders. Count = grants not revoked, role not archived, profile active, user active. Every change that could reduce it (revoke, archive role, disable or remove profile, deactivate or erase user) first takes `pg_advisory_xact_lock` on the tenant (several tenants: sorted by ID), then counts and refuses with `409 last_administrators` . These grants cannot expire.

15.4 Role edits

Creating, editing or archiving a custom role, or changing its permissions, follows 8.2 and 8.3: editors add only permissions they hold; privileged permissions need `role-permissions:manage-privileged`; conflicts are re-checked and `access_version` bumped for every holder; archiving revokes active grants.

16. Support access

A society administrator creates a `support_access_grant` for a named platform-tenant profile (reason, ticket, `read_only` or `impersonate`, at most 4 hours), used through pipeline step 3b. Emergency access without a grant: `PLATFORM_OWNER` only, MFA and passkey, read-only, 1 hour, administrators notified. Impersonation: no money actions, no access changes, no exports, both identities in `audit_event`.

17. Code layout

```
src/
├── Shared/
|   ├── Access/   PermissionProvider, PermissionDefinition, AccessAssertion,
AssertionResult, RoleScope, ScopedBy
|   └── Tenancy/  TenantContext (interface), ResolvedTenant,
TenantRequirement
├── Tenancy/
|   ├── Domain/         Tenant, TenantDatabase, TenantDomain, Plan, Feature,
Subscription, TenantJob
|   ├── Application/    CreateTenant, ChangeTenantStatus, ChangePlan,
UpdateSignupPolicy, provisioning and migration jobs,
|   |                   TenantWritableAssertion,
ArchivedTenantAccessAssertion, PlanFeatureAssertion
|   ├── Contracts/      TenantProvisionedV1, TenantStatusChangedV1,
TenantPlanChangedV1, TenantDeletedV1 ...
|   └── Infrastructure/ TenantResolveListener, TenantResolver, TenantContext
implementation, _tenant route loader,
|                       DBAL middleware, messenger middleware
└── Iam/
    ├── Domain/Identity/ User, UserProfile, UserAuthenticator,
ExternalIdentity, ApiKey
    ├── Domain/Access/   Permission, Role, UserRole, UserRoleAssignRole,
PermissionConflict, SupportAccessGrant,
```

```
    |                   UserRoleAssignmentPolicy, SystemGrantPolicy,
TwoAdministratorRule
    ├── Application/    RegisterUserInTenant, profile, grant and role
commands, EffectiveAccessLoader, AccessDecision,
    |                   PermissionCheck, ScopeCheck, AssertionRegistry,
catalogue sync, grant expiry job,
    |                   scope_reference projection, StepUpAssertion,
OwnRecordAssertion
    ├── Contracts/      UserRegisteredV1, UserProfileCreatedV1,
UserRoleGrantedV1 ...
    └── Infrastructure/ SecurityUser, user provider, ProfileLoginValidator,
JWT and API key handlers, AccessVoter,
                        scope filters, audit writer (DBAL), XML mappings,
API resources
```

18. API endpoints

Area	Endpoints	Permission or access
Tenant lookup	`GET /api/tenants/resolve?slug=`	Public, `_tenant = none`
Sign-up and join	`POST /api/auth/register`, `/api/auth/register/verify`; `POST /api/tenants/{slug}/join`; `POST /api/me/profiles`	Public (tenant required, rate-limited) / authenticated
Authentication	`POST /api/auth`, `/api/auth/mfa`, `/api/token/refresh`, `/api/logout`, `/api/password-reset`, `/api/verify-email`; passkey routes	Authentication tab
Me	`GET/PATCH /api/me`; `GET /api/me/profiles`; `GET/PATCH /api/me/profile`; `GET /api/me/access`; `/api/me/authenticators`, `/api/me/external-identities`, `/api/me/sessions`	Authenticated; step-up for credential changes

Area	Endpoints	Permission or access
Platform tenants	`POST/GET /api/platform/tenants`; `GET/PATCH /api/platform/tenants/{id}`; `POST /api/platform/tenants/{id}/status`; `GET /api/platform/tenant-jobs`; `GET/POST /api/platform/plans`	`_tenant = platform`; `tenants:create`, `tenants:read`, `tenants:update`, `tenants:manage`, `tenant-jobs:list`, `plans:manage`
Platform staff	`POST/GET /api/platform/profiles`; `/api/platform/profiles/{id}/roles`	`_tenant = platform`; `platform-profiles:manage`, `user-roles:create`, `user-roles:delete`
Tenant settings	`GET/PATCH /api/tenant`; `PATCH /api/tenant/signup-policy`; `GET/POST/DELETE /api/tenant/domains` (+ `/verify`)	`tenant:read`, `tenant:manage`, `tenant-domains:manage`
Profiles	`GET/POST /api/profiles`; `GET/PATCH/DELETE /api/profiles/{id}`	`profiles:list`, `profiles:create`, `profiles:read`, `profiles:update`, `profiles:delete`
Service accounts	`GET/POST /api/service-accounts`; `POST/DELETE /api/service-accounts/{profileId}/api-keys[/{keyId}]`	`service-accounts:manage`, `api-keys:manage` (step-up)
Roles and permissions	`GET /api/permissions`; `GET/POST /api/roles`; `GET/PATCH/DELETE /api/roles/{id}` (delete = archive); `PUT /api/roles/{id}/permissions`	`permissions:list`, `roles:manage`, `role-permissions:manage` (+ `role-permissions:manage-privileged`)
Assignment rules and conflicts	`GET/POST/DELETE /api/user-role-assign-roles`; `GET/POST/PATCH /api/permission-conflicts`	`user-role-assign-roles:manage`, `permission-conflicts:manage`

Area	Endpoints	Permission or access
Grants	`GET/POST /api/profiles/{id}/roles`; `DELETE /api/profiles/{id}/roles/{userRoleId}`	`user-roles:list`, `user-roles:create`, `user-roles:delete` (five gates)
Audit and support	`GET /api/audit-events`; `GET/POST/DELETE /api/support-access-grants`	`audit-events:read`, `support-access-grants:manage`

Creating endpoints accept an `Idempotency-Key` header.

19. Events

Event (V1)	Producer	Consumers
`TenantCreatedV1`, `TenantProvisionedV1`, `TenantStatusChangedV1`, `TenantPlanChangedV1`, `TenantSignupPolicyChangedV1`, `TenantDeletedV1`	Tenancy	IAM, business modules, Notification
`TenantJobFailedV1`	Tenancy	Monitoring
`UserRegisteredV1`, `UserDeactivatedV1`, `UserErasedV1`	IAM	Notification; business modules erase copies
`UserProfileCreatedV1`, `UserProfileUpdatedV1`, `UserProfileStatusChangedV1`	IAM	Business modules (profile-name projection), Notification
`UserRoleGrantedV1`, `UserRoleRevokedV1`, `ProfileAccessChangedV1`	IAM	Notification, Reporting
`RoleChangedV1`, `PermissionCatalogueChangedV1`	IAM	Monitoring

Event (V1)	Producer	Consumers
`UserSignedInV1`, `SignInFailedV1`, `SessionsRevokedV1`, `RefreshTokenReuseDetectedV1`, `AuthenticatorAddedV1`, `AuthenticatorRevokedV1`, `ExternalIdentityLinkedV1`, `ApiKeyCreatedV1`, `ApiKeyRevokedV1`	IAM	Notification, security monitoring
`SupportAccessGrantedV1`, `SupportAccessEndedV1`	IAM	Notification
`UnitActivatedV1`, `UnitClosedV1`, `VentureActivatedV1`, `VentureClosedV1` (versioned)	Business modules	IAM (`scope_reference`; closing revokes scoped grants)

Payloads carry IDs, not emails or phone numbers. Every IAM change also writes `audit_event`.

20. Errors and caching

Situation	HTTP	Code
Tenant missing, mismatched, unknown, unavailable	400 / 400 / 404 / 503	`tenant_required`, `tenant_mismatch`, `tenant_not_found`, `tenant_unavailable`
No active profile in the tenant	404	`tenant_not_found`
Tenant read-only	403	`tenant_read_only`
Missing permission or outside scope	403	`access_denied`
Assertion denied	403	the assertion's reason code
Step-up needed	401	`step_up_required`
Gate failure	403	`user_role_forbidden` (gate number)
Permission conflict	409	`permission_conflict`

Situation	HTTP	Code
Last administrators	409	`last_administrators`
Repeated request with a used idempotency key	original status	the original response

Caching follows section 14 step 4; assertions read live state; all keys are prefixed by tenant.

21. Out of scope for now

Topic	Plan
Approval workflow	Its own module; plugs in through assertions and defines its own permissions
Cooperative membership	Members, staff, employers; a profile will link to a member record
Governance	Elections, resolutions, access recertification
Temporal grant validity	PostgreSQL 18 `WITHOUT OVERLAPS` on a `tstzrange`, only if time-bound access becomes a product feature

22. Governance tests

Test	Asserts
`SchemaConventionTest`	Section 5: names, no native enums or `citext`, FK columns indexed, `timestamptz`, partial unique indexes
`ConstraintPresenceTest`	Composite FKs and every `_ck` trigger exist in `pg_constraint` and `pg_trigger`
`AuditPartitionTest`	`audit_event` key includes `occurred_at`; the application role cannot update or delete

Test	Asserts
`PermissionCatalogCoverageTest`	Every API operation has a catalogued slug or `_tenant = none` public access
`TenantRequirementTest`	Every route has `_tenant`; platform routes refuse society tenants
`SystemRoleCatalogueTest`	System roles use existing permissions within their scopes; no `money` or privileged permissions on `TENANT_ADMIN`
`UniquenessTest`	One profile per user and tenant; one active grant per profile, role and reference; no duplicate or shadowing role codes
`RoleTenantTriggerTest`	No grant, edge or copy references another tenant's role; platform roles only in the platform tenant
`SignUpTest`	Nothing created before verification; existing email gets a join link; removed profiles reactivate; disabled refused; policy and domain enforced
`SystemGrantTest`	Only `SELF_SERVICE` and bootstrap `TENANT_ADMIN` via the system path; gates 3 and 4 still run
`RoleEditTest`	Editors add only permissions they hold; privileged permissions gated; holders' access refreshed
`PermissionConflictTest`	Copied roles cannot escape conflicts; tenants cannot weaken defaults
`TwoAdministratorsTest`	Concurrent revoke and disable cannot drop below two
`GrantExpiryTest`	Expired grants stop within a minute and can be re-granted
`AccessDecisionTest`	Unanimous strategy; deny wins; challenge gives 401; reasons reach the response
`SupportAccessTest`	Grant works only for its tenant and window; impersonation limits
`CrossTenantTest`	No access, cache, API key or token-origin leaks between tenants

23. Checklist for new features

- [ ] New tables follow section 5; rules a foreign key cannot express are triggers with tests
- [ ] No fact stored twice; derived values computed; history in `audit_event`
- [ ] Every route declares `#[TenantRequirement]`; permissions declared in the module's `PermissionProvider`; every operation has a `security` expression; collections use scope filters
- [ ] Record-level rules written as assertions in the owning module
- [ ] Events listed in section 19 with schemas and no personal data; governance tests updated

24. Change history

Version	Change
v2	Clean rewrite after two reviews: one IAM module; platform tenant; permission-level conflicts; resolver at priority 20; pending sign-up; idempotent rejoin; system grant path; database-enforced tenant rules; partitioned audit; unanimous decisions. Review findings and sources: previous tab (section 1) and `doc/research_notes/data_model/`
v1	Profile pattern with separate Identity and Access modules (previous tab)

25. Table and field guide: what, why and how

This section explains every table and every field in plain terms: **what** it holds, **why** it exists, and **how** it is written and read. Sections 6–8 remain the precise definitions (types, constraints, indexes); this guide is for understanding them.

25.1 tenancy.tenant

What: one row per society, plus one row for Colsynq itself (the platform tenant). **Why:** everything in the system belongs to a tenant; this is the registry the resolver looks up on every request. **How:** created by platform staff (`POST /api/platform/tenants`), changed

by Tenancy commands, read through the Tenant Resolver and cached as `ResolvedTenant`.

Field	What it holds	Why it exists	How it is set and used
`id`	UUID v7 of the tenant	Stable key used everywhere (`tenant_id` columns, events, cache keys)	Generated by the application at creation; never changes
`kind`	`platform` or `society`	Colsynq staff need a tenant of their own so no profile has a null tenant	Set once; exactly one `platform` row; platform routes resolve only to it
`slug`	Short lower-case name, e.g. `ikeja-coop`	Human-friendly identifier for URLs and the `X-Tenant` header	Chosen at creation; validated by pattern; resolver matches `X-Tenant` and `<slug>.colsynq.app` against it
`legal_name`, `short_name`	Registered name and display name	Legal documents need the full name; screens need a short one	Entered at creation; editable by platform staff and tenant admins
`registration_number`, `registering_authority`, `state`	Cooperative registration details	Proves the society is a registered cooperative; needed for compliance and reports	Entered at creation; null for the platform tenant
`status`	Lifecycle state	Decides whether the tenant can be reached, read or written	Changed only by Tenancy commands and jobs; checked by the resolver and `TenantWritableAssertion`
`status_reason`	Why the status was set	Officers and support must see why a society is suspended, archived or failed	Required by CHECK for those statuses

Field	What it holds	Why it exists	How it is set and used
`status_changed_at`	When the status last changed	Shows how long a society has been in a state	Set with every status change; full history is in `audit_event`
`activated_at`	First time the tenant became active	Billing and reporting start from this date	Set once by the provisioning job
`signup_mode`	`open`, `email_domain` or `closed`	Each society decides who may join by themselves	Set by tenant admins (`PATCH /api/tenant/signup-policy`); read by sign-up via `TenantContext`
`signup_email_domains`	Allowed email domains	Lets a society accept only its employer's staff emails	Required only for `email_domain` ; compared with the verified email
`require_email_verification`	Whether new users must verify email before signing in	Stops fake or mistyped addresses; required for `email_domain`	Default true; CHECK forces it for `email_domain`
`region` , `timezone` , `currency`	Locale settings	Dates and amounts display correctly	Defaults `ng` , `Africa/Lagos` , `NGN` ; passed to modules via `TenantContext`
`logo_url` , `primary_colour`	Branding	The society's own look on sign-in and sign-up pages	Returned by the public lookup `GET /api/tenants/resolve`
`created_by_profile_id`	The platform staff profile that created it	Accountability	Set at creation; logical link to `iam.user_profile`
`created_at` , `updated_at`	Row timestamps	Standard housekeeping	Set by the application

25.2 tenancy.tenant_database

What: where a society's own database lives and how to connect to it. **Why:** each society has a separate database (Multi-tenancy tab); this is the map from tenant to database. Kept apart from `tenant` because it is more sensitive and changes more often. **How:**

written by provisioning and migration jobs; read by the Tenant Resolver to switch the database connection.

Field	What it holds	Why it exists	How it is set and used
`tenant_id`	The society	One database per society	Primary key and FK to `tenant`
`db_host`, `db_port`	Database server address	Societies can live on different servers for load and moves	Chosen by the provisioning job; updated when a tenant is moved
`db_name`	Database name	Identifies the society's database on the server	Generated, never typed by people
`db_role`	Database login role for this society	Limits the damage if one society's credentials leak	Created by the provisioning job
`secret_ref`	Pointer to the password in the vault	Passwords must never sit in the landlord database	Read at connection time from the vault
`schema_version`	Last migration applied	Lets operators see which societies are behind without opening each database	Copied from the tenant database after each migration
`schema_status`	`ok`, `migrating`, `failed`, `drift`	Shows migration health per society	Set by `tenants:migrate` and the drift check
`migration_wave`	0–3	Migrations roll out to canary societies first	Set by platform staff
`provisioned_at`	When the database was created	Audit and support	Set once by the provisioning job

25.3 tenancy.tenant_domain

What: custom web addresses a society uses, e.g. `portal.ikejacoop.ng`. **Why:** some societies want their own domain; subdomains of colsynq.app are derived from the slug and need no row. **How:** tenant admins add a domain, prove ownership through DNS, and the resolver then accepts it.

Field	What it holds	Why it exists	How it is set and used
`host`	The domain name	The resolver matches incoming hosts against it	Primary key; entered by tenant admins
`tenant_id`	Owning society	Maps the domain to a tenant	Set when added
`is_primary`	Main domain flag	Emails and links use one canonical address	At most one per tenant (partial unique index)
`verification_token_hash`	Hash of the DNS verification token	Proves the society controls the domain without storing the token	Token shown once; hash compared when DNS is checked
`verified_at`	When ownership was proven	Unverified domains are never used for resolution	Set by `/verify`

25.4 tenancy.feature, plan, plan_feature

What: the product catalogue: features (`procurement`, `ventures`, …), plans, and which features and limits each plan includes. **Why:** what a society may use depends on its plan; storing this as rows makes it queryable and validated. **How:** platform staff manage plans; `PlanFeatureAssertion` reads the current plan's features through `TenantContext`.

Table and field	What it holds	Why it exists	How it is set and used
`feature.code`	Feature key	Permissions point to a feature (`permission.feature_code`)	Defined with the product; primary key
`feature.name`, `description`	Labels	For plan pages and admin screens	Edited by platform staff
`plan.id`	Plan version ID	Subscriptions point to an exact version	Generated
`plan.code`, `version`	Plan name and version	Changing a plan creates a new version so existing subscribers are unaffected	Unique together
`plan.name`	Display name	Shown to customers	Edited by platform staff
`plan.is_active`	Whether new subscriptions may choose it	Old versions stay for existing subscribers	Set by platform staff
`plan_feature.plan_id`, `feature_code`	A feature included in a plan	Defines what each plan offers	Composite primary key
`plan_feature.limit_value`	Numeric cap (users, units, storage)	Some features are limited by quantity	Null = unlimited; checked by the module that owns the counted thing

25.5 tenancy.subscription

What: which plan a society is on, and when. **Why:** the current plan must have one source of truth; history of plans is kept. **How:** created at tenant creation and on plan changes; exactly one current row per tenant.

Field	What it holds	Why it exists	How it is set and used
`id`	Subscription ID	Reference for billing	Generated
`tenant_id`	The society	Links the plan to the tenant	Set at creation
`plan_id`	Exact plan version	Fixes the features and limits that apply	Set at creation or change
`status`	`trialing`, `active`, `past_due`, `cancelled`, `expired`	Billing state affects access (e.g. suspension)	Changed by billing; one current row enforced by a partial unique index
`starts_at`, `ends_at`	Period	Renewal and expiry	Set by billing

25.6 tenancy.tenant_entitlement_override

What: one-off exceptions to a plan for a single society. **Why:** sales deals or pilots sometimes enable a feature or raise a limit without changing the plan. **How:** platform staff add a row; `PlanFeatureAssertion` checks overrides before the plan.

Field	What it holds	Why it exists	How it is set and used
`tenant_id`, `feature_code`	The society and feature	One override per feature per society	Composite primary key
`is_enabled`	Turn the feature on or off	Grants or withholds a feature regardless of plan	Set by platform staff
`limit_value`	Custom limit	Raises or lowers a plan limit	Null = plan limit applies

Field	What it holds	Why it exists	How it is set and used
`expires_at`	End of the exception	Pilots and promotions end automatically	Ignored after this time
`reason`	Why it was granted	Accountability	Required text

25.7 tenancy.tenant_job

What: a log of long-running operations on a tenant: provisioning, migrations, moves, archiving, deletion. **Why:** these run in the background and can fail halfway; operators need status, errors and retries in one place. **How:** written by workers; a partial unique index on running jobs prevents two jobs on the same tenant at once.

Field	What it holds	Why it exists	How it is set and used
`id`	Job ID	Reference for logs and retries	Generated
`tenant_id`	Target society	Jobs act on one tenant	Set when queued
`kind`	`provision`, `migrate`, `move`, `archive`, `delete`	One table for every job type	Set when queued
`rollout_id`	Fleet rollout this job belongs to	Groups the per-tenant jobs of one migration rollout	Set by `tenants:migrate`; indexed
`step`	Current step	Resume from the failed step	Updated by the worker
`status`	`queued`, `running`, `succeeded`, `failed`	Progress and the per-tenant lock	Only one `running` per tenant
`attempt`	Retry count	Stop after repeated failures	Incremented on retry
`from_version`, `to_version`	Schema versions for migrations	Shows what a migration changed	Set for `migrate` jobs

Field	What it holds	Why it exists	How it is set and used
`details`	Extra job data	Varies by job type	jsonb; never used for filtering
`error`	Last error message	Diagnosis	Set on failure
`started_at`, `finished_at`	Timing	Duration and monitoring	Set by the worker

25.8 iam.app_user (entity User)

What: one row per person or system that can sign in to Colsynq, across all societies. **Why:** a person has one login even if they belong to several societies; society-specific data lives on profiles. **How:** created by sign-up verification, by administrators, at tenant bootstrap or for service accounts; read at every request by the firewall (`SecurityUser`).

Field	What it holds	Why it exists	How it is set and used
`id`	UUID v7 of the user	The only identifier put in tokens (`sub`)	Generated; never changes
`kind`	`person` or `service`	Service accounts (integrations) follow different rules: no password, no MFA, one profile	Set at creation; never changes
`email`	Normalised email address	Sign-in identifier and contact for persons	Set at sign-up or by an admin; unique; changes require verification
`phone`	Phone number in E.164 format	SMS one-time codes and contact	Optional; unique only once verified, because families share phones

Field	What it holds	Why it exists	How it is set and used
`password_hash`	Argon2id hash	Password sign-in	Set when the user chooses a password; null for service and passkey-only users; re-hashed on login when the algorithm improves
`status`	`active` or `deactivated`	Lets an account be switched off everywhere without deleting it	Changed by the user (close account), platform staff or erasure
`token_version`	Integer copied into the JWT `ver` claim	"Sign out everywhere" without a token database	Bumped on password change, sign-out everywhere, deactivation; tokens with an older value are refused
`failed_login_count`	Consecutive failed sign-ins	Slows down password guessing	Incremented on failure, reset on success
`locked_until`	End of a temporary lock	Back-off after repeated failures, measured in minutes so attackers cannot lock people out for long	Set by the login failure handler; "locked" means this time is in the future

Field	What it holds	Why it exists	How it is set and used
`email_verified_at`, `phone_verified_at`	When each was proven	Unverified contacts cannot be trusted for sign-in, recovery or domain checks	Set by verification links or codes
`trusted_device_version`	Integer for remembered devices	"Forget all my devices" in one step	Bumped by the user or after a password reset
`last_login_at`	Last successful sign-in	Shown to the user; dormant-account reports	Set on sign-in
`created_at`, `updated_at`	Row timestamps	Housekeeping	Set by the application

25.9 iam.user_profile (entity UserProfile)

What: a user's membership of one society, with how the society knows them. **Why:** the profile is how we know which societies a user belongs to; roles are granted to the profile, so access is always society-specific. **How:** created automatically by `RegisterUserInTenant`; read on every request to check the user may enter the resolved tenant.

Field	What it holds	Why it exists	How it is set and used
`id`	UUID v7 of the profile	Grants, API keys and audit refer to it	Generated
`user_id`	The User	Links the society membership to the login	Set at creation
`tenant_id`	The society	Every profile belongs to exactly one tenant	Set at creation; never changes

Field	What it holds	Why it exists	How it is set and used
`first_name`, `last_name`	Name as the society records it	Societies may record names differently	Entered at sign-up or by an admin
`status`	`active`, `disabled`, `removed`	Admins can pause or end a member's access without deleting history	Changed by admins or when the user leaves; checked on every request
`status_reason`, `status_changed_at`	Why and when	Accountability	Set with each status change
`source`	How the profile first came to exist	Reporting and abuse investigation	Set once: `self_signup`, `admin_created`, `tenant_bootstrap`, `join_link`
`access_version`	Integer used in the access cache key	Changes to access take effect on the next request	Bumped whenever the profile's grants, status or its roles' permissions change
`created_by_profile_id`	Admin profile that created it	Accountability	Set for admin-created and bootstrap profiles; null for self sign-up
`metadata`	Free-form society data (staff number, branch, next of kin, …)	Keeps the module generic while letting each society collect what it needs	Written by the society's screens; never used for sign-in, access or uniqueness
`created_at`, `updated_at`	Row timestamps	Housekeeping	Set by the application

25.10 iam.user_authenticator and iam.backup_code

What: the second factors a user has set up: authenticator apps (TOTP), email codes, passkeys, and backup codes. **Why:** NIST SP 800-63B treats these as "authenticators"; one table with a type keeps their lifecycle (add, use, revoke) identical. **How:** the user enrols them under `/api/me/authenticators`; sign-in and step-up read them.

Field	What it holds	Why it exists	How it is set and used
`id`	Authenticator ID	Manage each factor separately	Generated
`user_id`	Owner	Factors belong to the user, not a profile	Set at enrolment
`type`	`totp`, `email_otp`, `passkey`, `backup_codes`	Different verification logic per kind	Set at enrolment
`label`	User's name for it ("Work phone")	Users tell their devices apart	Entered by the user
`credential_id`	Passkey credential ID	Browsers identify passkeys by this	Set at passkey registration; unique
`public_data`	Passkey public key, sign count, transports, device type	Needed to verify passkey sign-ins	Written at registration; sign count updated on each use

Field	What it holds	Why it exists	How it is set and used
`secret_encrypted`	Encrypted TOTP seed	Needed to check app codes; encrypted so a database leak does not expose it	Written at enrolment
`confirmed_at`	When enrolment was confirmed	Unconfirmed factors are not used	Set after the first correct code
`last_used_at`	Last successful use	Spot unused or suspicious factors	Updated on use
`revoked_at`	When removed	Removed factors stop working; history stays	Set when the user or support removes it
`backup_code.authenticator_id`	The backup-code set	Groups the codes	Set when codes are generated
`backup_code.code_hash`	Hash of one code	Codes are passwords; only hashes are stored	Shown once in plain text at generation
`backup_code.used_at`	When used	Each code works once	Set on use

25.11 iam.external_identity

What: links between a Colsynq User and an outside sign-in (Google, Microsoft, Apple, a future identity provider, or an OAuth client). **Why:** lets people sign in with another provider and lets service accounts use `client_credentials` later. **How:** linked from a signed-in session or a verified sign-up; read when an outside sign-in arrives.

Field	What it holds	Why it exists	How it is set and used
`id`	Link ID	Unlink individually	Generated
`user_id`	Colsynq User	The account the outside identity signs into	Set when linked
`provider`	e.g. `google`, `oidc:<issuer>`, `client:<issuer>`	Distinguishes sources	Set when linked
`subject`	The provider's permanent user or client ID	Matching is by this, never by email, to prevent account takeover	Unique with `provider`
`email`, `email_verified`	What the provider asserted	Shown to the user; never trusted alone	Updated on each sign-in
`linked_at`, `last_used_at`, `unlinked_at`	Lifecycle times	Audit and cleanup	Set by link, sign-in and unlink

25.12 iam.refresh_token

What: long-lived tokens used to get new short-lived access tokens. **Why:** access tokens last 10 minutes; refresh tokens keep people signed in safely. **How:** issued at sign-in, replaced on every use (rotation), revoked on sign-out (Authentication tab section 8).

Field	What it holds	Why it exists	How it is set and used
token	Hash of the refresh token	The real token is never stored	Bundle setting `hash_tokens` enabled
`username`	User ID	Links the token to its User	Set at issue
`valid`	Expiry time	Tokens stop working after 14 days	Set at issue

Field	What it holds	Why it exists	How it is set and used
`family`, `family_valid`	Chain of rotated tokens	If an old token is reused, the whole chain is revoked (theft detection)	Managed by the bundle
`origin`	Society host that issued it	A token stolen on one society's site cannot be refreshed from another	Set at issue; checked on refresh

25.13 iam.api_key

What: keys used by integrations (payroll uploads, partner systems) instead of passwords. **Why:** machines need credentials that can be scoped, rotated and revoked without a person's login. **How:** created by tenant admins for a service account; checked on each API call carrying `Authorization: ApiKey …`.

Field	What it holds	Why it exists	How it is set and used
`id`	Key ID	Manage keys individually	Generated
`user_profile_id`, `tenant_id`	The service account's profile and its society	The key acts as that profile, only in that society	Composite FK keeps them consistent; key refused in any other tenant
`name`	Admin's label	Tell keys apart	Entered by the admin
`prefix`	First characters of the key	Recognise a key in logs without revealing it	Stored at creation
`lookup_hash`	HMAC-SHA-256 of the key with a server secret	Find the key quickly without storing it	Computed at creation and on each call
`ip_allow_list`	Allowed network addresses	Limits where a leaked key can be used	Optional; checked on each call

Field	What it holds	Why it exists	How it is set and used
`expires_at`	Expiry (max 1 year)	Keys must be rotated	Set at creation
`last_used_at`	Last call	Find unused keys	Updated on use
`revoked_at`	When revoked	Instant shut-off	Set by admins or when the profile is removed
`created_at`	Creation time	Housekeeping	Set by the application

25.14 iam.one_time_token

What: single-use secrets for pending sign-ups, password resets and first-time "set your password" links. **Why:** these links must expire, work once, and not reveal the secret if the database leaks. **How:** a selector finds the row; the verifier is checked against its hash.

Field	What it holds	Why it exists	How it is set and used
`id`	Token row ID	Housekeeping	Generated
`purpose`	`signup`, `password_reset`, `set_password`	One table for every one-time link	Set at creation
`selector`	Public half of the link	Find the row without exposing the secret	Unique
`verifier_hash`	Hash of the secret half	Proves the link is genuine	Compared on use
`user_id`	User concerned	Reset and set-password links act on a user	Null for `signup` (no user exists yet)
`tenant_id`	Society for a pending sign-up	The account is created in this tenant once verified	Set for `signup`

Field	What it holds	Why it exists	How it is set and used
`payload_encrypted`	Encrypted pending sign-up data (names, password hash)	Nothing is created until the email is proven (prevents pre-registration attacks)	Decrypted and used when the link is opened
`expires_at`	Expiry	Links stop working after a set time	Set at creation
`used_at`	When used	Single use	Set on use
`created_at`	Creation time	Rate limiting and cleanup	Set by the application

25.15 What is deliberately not stored

Item	Where instead	Why
Email-verification and join links	Signed URLs	They need no server state
JWT blocklist, refresh-reuse cache	Redis	Short-lived, high-volume data
MFA enabled, platform staff, locked, verified	Computed from other fields	Avoids two sources of truth
Society-specific contact details and member numbers	`user_profile.metadata` now; the Membership module later	Keeps IAM generic

25.16 iam.permission (catalogue)

What: every action the system can protect, such as `purchase-orders:create`. **Why:** permissions are the smallest unit of access; one global list keeps them consistent across societies. **How:** declared in code by each module (`PermissionProvider`) and copied in by `iam:catalogue:sync` on deploy; nobody edits them by hand.

Field	What it holds	Why it exists	How it is set and used
`slug`	`resource:action` name	The name used in `security` expressions and in roles	Primary key; written by the sync
`resource`, `action`	The two halves of the slug	Filter the role editor by resource or action	CHECK keeps them equal to the slug
`module`	Owning module	Shows where a permission comes from	From the provider
`name`, `description`	Readable labels	Admins choose permissions in the role editor	From the provider
`risk`	`low`, `high`, `money`	Higher-risk permissions need MFA, step-up and privileged editors	From the provider
`allowed_scopes`	Scopes it may be granted at	A unit-level permission cannot be put in a platform role, and so on	From the provider; checked when added to a role
`feature_code`	Plan feature it belongs to	Features not in the plan are blocked	From the provider; checked by `PlanFeatureAssertion`
`deprecated_at`	When removed from code	Gives one release to migrate before deletion	Set by the sync; the next release deletes it and its links

25.17 iam.role (catalogue)

What: named bundles of permissions. System roles are shared by every society; custom roles belong to one society. **Why:** granting roles is simpler and safer than granting single permissions. **How:** system roles come from code (`SystemRoleCatalogue`); custom roles are created by tenant admins.

Field	What it holds	Why it exists	How it is set and used
`id`	Role ID	Grants and edges refer to it	Generated
`tenant_id`	Owning society, or null for a system role	Separates shared roles from society-specific ones	Set at creation; cannot change
`code`	Short key, e.g. `CASHIER`	Stable reference in code, seeds and APIs	Unique per tenant; a custom role cannot reuse a system code
`name` , `description`	Labels	Shown in admin screens	Editable for custom roles
`scope`	Where it can be granted: platform, whole society, unit, venture or self	Decides whether a grant needs a unit or venture reference	Set at creation; cannot change
`can_assign_all`	Wildcard assignment right	Tenant administrators can grant any non-platform role without listing every edge	Set on system roles; rarely on custom ones
`based_on_role_id`	Role it was copied from	Shows origin; makes upgrades easier to review	Set when copying
`max_grant_duration`	Longest allowed grant	Temporary roles (e.g. external auditor) must expire	When set, grants need `expires_at` within it

Field	What it holds	Why it exists	How it is set and used
`archived_at`	When retired	Retire a role without losing history	Archiving revokes its active grants
`created_at`, `updated_at`	Row timestamps	Housekeeping	Set by the application

25.18 iam.role_permission

What: which permissions a role contains. **Why:** the many-to-many link between roles and permissions. **How:** set by the catalogue sync for system roles, and by authorised editors for custom roles.

Field	What it holds	Why it exists	How it is set and used
`role_id`	The role	Side one of the link	Part of the primary key
`permission_slug`	The permission	Side two of the link	Part of the primary key; the editor must hold it; risky ones need the privileged permission
`created_at`	When added	Audit	Set on insert; changes bump every holder's `access_version`

25.19 iam.user_role (grant)

What: a role given to one profile, optionally limited to one unit or venture, optionally for a limited time. **Why:** this is what actually gives a person access in a society. **How:** created through the five gates (or the system path); read on every request to build the user's effective access.

Field	What it holds	Why it exists	How it is set and used
`id`	Grant ID	Revoke individually	Generated

Field	What it holds	Why it exists	How it is set and used
`user_profile_id`	Who receives the role	Grants belong to a society membership, not a global user	Set at grant
`tenant_id`	The profile's society	Lets the database prove the grant and profile are in the same society, and speeds up per-society queries	Must equal the profile's tenant (composite FK)
`role_id`	Which role	What is granted	Must be a system role or this society's, not archived (trigger)
`scope_ref_id`	Which unit or venture	Unit and venture roles apply to one place	Required for unit and venture roles, null otherwise; must exist in `scope_reference`
`granted_at`	When granted	Audit and display	Set at grant
`expires_at`	When it ends	Temporary access ends without anyone remembering to revoke it	Required within the role's `max_grant_duration`; the expiry job revokes it
`revoked_at`	When it ended	Rows are kept for history	Set by revoke, expiry, profile removal or role archive

25.20 iam.user_role_assign_role

What: rules saying which roles a holder of a given role may grant (e.g. a Unit Manager may grant Cashier). **Why:** lets managers run their own units without being society administrators (delegated administration, ARBAC97). **How:** default rules ship with the catalogue; societies add their own `allow` or `deny` rows.

Field	What it holds	Why it exists	How it is set and used
`id`	Rule ID	Manage individually	Generated

Field	What it holds	Why it exists	How it is set and used
`tenant_id`	Null for a default rule, or the society	Defaults for everyone; overrides per society	Set at creation
`assigner_role_id`	Role A (who may assign)	The granting side	Must be a system role or this society's
`assignable_role_id`	Role B (what may be assigned)	The granted side; may equal A for deputies	Same check
`effect`	`allow` or `deny`	A society can switch off a default rule	Precedence: tenant deny, wildcard, tenant allow, default allow
`created_at`	When added	Audit	Set by the application

25.21 iam.permission_conflict

What: pairs of permissions one person should not hold together (e.g. paying and reconciling). **Why:** separation of duties prevents one person from committing and hiding fraud; checking permissions (not roles) means copies of roles cannot escape the rule. **How:** defaults ship with the catalogue; societies may add stricter rules; checked whenever a grant or a role's permissions change.

Field	What it holds	Why it exists	How it is set and used
`id`	Rule ID	Manage individually	Generated
`tenant_id`	Null for a default, or the society	Defaults for everyone; societies add their own	Set at creation
`permission_a`, `permission_b`	The two permissions	The conflicting pair	Stored in alphabetical order so each pair exists once

Field	What it holds	Why it exists	How it is set and used
`same_scope_only`	Whether the conflict applies only in the same unit or venture	A cashier in one unit may be an accountant in another	Set with the rule
`severity`	`block` or `warn`	Some pairs are forbidden, others need a written reason	A society may only make a default stricter

25.22 iam.platform_tenant

What: a single row naming which tenant is Colsynq's platform tenant. **Why:** database triggers must know it to stop platform roles being granted in societies, and IAM does not read Tenancy's tables. **How:** written once at bootstrap; read by triggers.

Field	What it holds	Why it exists	How it is set and used
`singleton`	Always true	Guarantees exactly one row	Primary key with a CHECK
`tenant_id`	The platform tenant's ID	Used by triggers	Set at bootstrap

25.23 iam.support_access_grant

What: time-boxed permission for a Colsynq support person to look into one society. **Why:** support needs access to fix problems, but staff must have no standing access to society data. **How:** created by a society administrator; used through request pipeline step 3b.

Field	What it holds	Why it exists	How it is set and used
`id`	Grant ID	Reference in audit	Generated
`tenant_id`	The society	Access is for one society only	Set by the granting admin's context

Field	What it holds	Why it exists	How it is set and used
`support_profile_id`	Support person's platform profile	Who may enter	Must be a platform-tenant profile (trigger)
`granted_by_profile_id`	Society admin who allowed it	Accountability	Must be a profile of the same society
`mode`	`read_only` or `impersonate`	Most support needs only to look	Chosen by the admin
`reason`, `ticket_ref`	Why and which support ticket	Accountability	Required
`starts_at`, `expires_at`	Time window (max 4 hours)	Access ends automatically	CHECK enforces the window
`revoked_at`	Early end	Admins can cut access at once	Set by the admin

25.24 iam.scope_reference and iam.audit_event

scope_reference — What: IAM's copy of the business units and ventures that exist in each society. **Why:** unit and venture IDs live in society databases; IAM needs them to validate and filter scoped grants without reading those databases. **How:** filled from unit and venture events.

Field	What it holds	Why it exists	How it is set and used
`tenant_id`, `scope_ref_id`	Society and unit or venture ID	Identify the reference	Composite primary key; target of grants' foreign key
`kind`	`business_unit` or `venture`	Must match the role's scope	From the event
`name`	Display name	Shown next to scoped grants	From the event

Field	What it holds	Why it exists	How it is set and used
`status`	Active or closed	Closed units cannot receive grants; closing revokes them	From the event
`source_version`	Event version	Ignore events that arrive out of order	Applied only when newer
`updated_at`	Last update	Monitoring	Set on apply

audit_event — What: an append-only record of every IAM change and security event. **Why:** answers "who did what, when and why" for compliance and investigations, and keeps that history out of the working tables. **How:** written in the same transaction as each change; only read, never edited; old monthly partitions are dropped after 7 years.

Field	What it holds	Why it exists	How it is set and used
`occurred_at`, `id`	Time and event ID	Together the primary key, as partitioning requires	Set on insert
`tenant_id`	Society concerned	Per-society audit views	Null for platform-wide events
`actor_user_id`, `actor_profile_id`	Who acted	Accountability	Null for the system
`actor_kind`	`user`, `system` or `support`	Separates human, automatic and support actions	Set on insert
`action`	What happened, e.g. `user_role.granted`	Searchable event type	Set on insert
`subject_type`, `subject_id`	What it happened to	Find all events for one record	Indexed
`payload`	Before, after, reason	Details without personal data (IDs only)	jsonb

Field	What it holds	Why it exists	How it is set and used
`ip`, `user_agent`	Request origin	Security investigations	From the request
`correlation_id`	Request or message ID	Link the audit entry to logs and events	From the request context

26. How JWT, refresh tokens and OAuth2 work: tables and operations

This section follows each token through its life and names every table, cache and key it touches. Configuration details are in the Authentication tab (sections 7–11).

26.1 The three credentials at a glance

Credential	What it is	Lifetime	Where it is kept	What proves it is valid
Access token (JWT)	A signed, self-contained statement "this is user X"	10 minutes	Client memory only; **not stored** by Colsynq	ES256 signature with the platform key (vault), plus `iss`, `aud`, `exp`, `ver` checks
Refresh token	A random opaque string used to get new access tokens	14 days, rotated on every use, 30-day cap	Client: httpOnly cookie (web) or secure storage (mobile). Server: **hash** in `iam.refresh_token`	Hash lookup in `iam.refresh_token`, not expired, family still valid, origin matches
API key	A random secret for a service account	Up to 1 year	Server: HMAC hash in `iam.api_key`	Hash lookup, not revoked or expired, tenant and IP match

26.2 Anatomy of the access token

Part	Content	Why
Header	`alg: ES256`, `typ: at+jwt`, `kid` (key ID)	Fixed algorithm (no `alg: none` or swaps); `kid` picks the current or previous public key during rotation
`sub`	`app_user.id`	Who the token belongs to; never the email
`iss`, `aud`	`https://api.colsynq.app`, `colsynq-api`	Refuse tokens made for another system
`iat`, `exp`	Issue and expiry time (10 minutes)	Short life limits damage if stolen
`jti`	Random token ID	Lets one token be blocked at logout
`ver`	`app_user.token_version` at issue time	"Sign out everywhere" without storing tokens
`amr`, `auth_time`	Methods used (`pwd`, `otp`, `hwk`) and when	Step-up and per-tenant MFA policy
Not included	Email, names, tenant, profile, roles, permissions	Personal data stays out; access is decided on the server per request and per society

The signing keys are not in a table: the private key lives in the vault, and old public keys stay listed for one token lifetime during rotation.

26.3 Operations and the tables they touch

Operation	Reads	Writes	Cache / other
Sign in with password (`POST /api/auth`)	`app_user` (email, `password_hash`, `status`, `locked_until`, `email_verified_at`); `user_profile` when a tenant is resolved (`ProfileLoginValidator`)	`app_user.failed_login_count`, `locked_until` (on failure) or `last_login_at` and rehashed `password_hash` (on success); new `refresh_token` row; `audit_event` (`user.signed_in` or `user.sign_in_failed`)	Login throttling counters in Redis

Operation	Reads	Writes	Cache / other
Second factor (`POST /api/auth/mfa`)	`user_authenticator` (TOTP seed, passkey public key); `backup_code`	`user_authenticator.last_used_at` and passkey sign count; `backup_code.used_at` ; `audit_event`	Used-code cache in Redis (no replay)
Call an API with the JWT	No token table. `app_user` (`status` , `token_version`); `user_profile` for the tenant; `user_role` , `role` , `role_permission` on a cache miss	None	`jti` blocklist (Redis); access cache keyed by `access_version`
Refresh (`POST /api/token/refresh`)	`refresh_token` by hash; `app_user` (`status`)	Old row marked used; new row in the **same family**; `audit_event`	Reuse cache (Redis); previous JWT's `jti` blocklisted
Refresh with an already-used token (theft signal)	`refresh_token` family	Whole family marked invalid (`family_valid`); `audit_event` `refresh_token.reuse_detected`	All the family's tokens refused; user notified
Log out (`POST /api/logout`)	`refresh_token`	That refresh token invalidated; `audit_event`	Current `jti` added to the blocklist until it expires
Sign out everywhere / password change / reset	`app_user` ; `one_time_token` (for reset)	`app_user.token_version + 1` (and new `password_hash`); all the user's `refresh_token` rows invalidated; `one_time_token.used_at` ; `audit_event`	Every existing JWT now fails the `ver` check
Deactivate user	`app_user`	`status = deactivated` , `token_version + 1` ; refresh tokens and API keys revoked; grants revoked; `audit_event`	Access caches become unreachable (new `access_version`)
Disable a profile in one society	`user_profile`	`user_profile.status` , `access_version + 1` ; `audit_event`	Tokens stay valid; that society now returns 404
Call with an API key	`api_key` by `lookup_hash` ; `user_profile` (service account, same tenant)	`api_key.last_used_at`	Access cache as for users
Expired or revoked access token	None	None	Client uses its refresh token, or signs in again

26.4 Sign-in, use and refresh: sequence

```
sequenceDiagram
    autonumber
    participant C as Client
    participant AUTH as POST /api/auth
    participant DB as iam tables
    participant R as Redis
    participant API as Any API endpoint
    participant RF as POST /api/token/refresh
```

```
    C->>AUTH: email + password (+ X-Tenant)
    AUTH->>DB: read app_user, verify hash, check user_profile in tenant
    AUTH->>DB: insert refresh_token (hash, family F1), audit_event
    AUTH-->>C: JWT (10 min, ver = token_version) + refresh token
    C->>API: Bearer JWT
    API->>R: jti blocked? access cache
    API->>DB: app_user token_version and status; profile; grants on cache
miss
    API-->>C: 200
    Note over C,API: 10 minutes later the JWT expires
    C->>RF: refresh token (cookie or body)
    RF->>DB: find by hash, check expiry, family valid, origin
    RF->>DB: mark old used, insert new token in family F1, audit_event
    RF->>R: remember spent token (reuse detection), block old jti
    RF-->>C: new JWT + new refresh token
```

26.5 Why this design

- **No token table for access tokens:** verifying a signature is fast and needs no database. Revocation is still possible through `token_version` (all tokens of a user), the Redis `jti` blocklist (one token) and the 10-minute life.
- **Refresh tokens are stored, hashed:** they live long, so they must be revocable one by one and detectable when stolen; hashing means a database leak does not leak usable tokens.
- **Rotation and families:** each refresh token works once. If an attacker and the real user both try the same token, the second use reveals theft and the whole family is cut.
- **Origin binding:** a refresh token issued on one society's site cannot be used from another society's site.
- **Tenant-free tokens:** one sign-in covers all of a user's societies; the profile check on each request decides which society they may enter right now.

26.6 OAuth2 in Colsynq: three roles

OAuth2 is a standard for letting one system obtain tokens from another. Colsynq can play three parts; only the first is used now.

Role	Meaning	When	Tables
OAuth client (social sign-in)	Colsynq sends the user to Google, Microsoft or Apple and receives proof of who they are	Now (optional per deployment)	`external_identity`, `app_user`, `user_profile`, `refresh_token`, `audit_event`
Resource server	Colsynq accepts access tokens issued by an external identity provider (Keycloak or similar)	Later, when partners or banks need delegated access	`external_identity` (maps the provider's `sub` to a User); JWKS keys cached, no table
Authorization server	Colsynq itself issues OAuth tokens to third-party apps	Not planned; an external identity provider is preferred (Authentication tab 10.2)	Would add client, scope, code and consent tables; not part of this design

26.7 Social sign-in (authorization code with PKCE)

```
sequenceDiagram
    autonumber
    participant B as Browser
    participant API as Colsynq API
    participant P as Provider (Google, Microsoft, Apple)
    participant DB as iam tables
    participant R as Redis
    B->>API: GET /api/auth/oauth/google/start (+ X-Tenant or society host)
    API->>R: store state + PKCE verifier (5 minutes)
    API-->>B: redirect to provider with state + code_challenge
    B->>P: user signs in at the provider
    P-->>B: redirect back with code + state
    B->>API: GET /api/auth/oauth/google/callback?code&state
    API->>R: check and delete state, get PKCE verifier
    API->>P: exchange code + verifier for tokens
```

```
    P-->>API: id_token (validated: signature, issuer, audience, nonce,
expiry)
    API->>DB: external_identity by (provider, subject)
    alt linked identity found
        API->>DB: load app_user; check user_profile in the tenant
    else no link yet
        API->>DB: new person: app_user (email verified by provider) +
user_profile via RegisterUserInTenant, if the tenant's sign-up policy allows
        API->>DB: existing email: nothing linked automatically; owner must
link while signed in
    end
    API->>DB: insert refresh_token, audit_event
    API-->>B: Colsynq JWT + refresh token (the provider's tokens are
discarded)
```

Step	Table or store	What happens
Start	Redis	`state` and PKCE verifier saved for 5 minutes (protects against forged callbacks and code theft)
Callback	Redis	`state` checked and deleted (single use)
Identity lookup	`external_identity`	Match on `(provider, subject)`, **never on email**, so nobody can take over an account by creating a provider account with the same email
New person	`app_user`, `user_profile`, `user_role`	Created through `RegisterUserInTenant` (section 13) with `SELF_SERVICE`; email counts as verified only if the provider verified it and owns the domain
Linking an existing account	`external_identity`	Only from a signed-in session (`POST /api/me/external-identities`), with step-up
Session	`refresh_token`, `audit_event`	Colsynq issues its own tokens; provider tokens are not stored

Step	Table or store	What happens
Use later	`external_identity.last_used_at`	Updated on each social sign-in

26.8 External identity provider as token issuer (later)

When an identity provider issues the access tokens, Colsynq stops issuing JWTs for those users and only validates:

Step	Table or store	What happens
Validate token	JWKS cache (no table)	Symfony `oidc` handler checks signature against the provider's published keys, `iss`, `aud`, `exp`, algorithm
Find the user	`external_identity` (`provider = oidc:<issuer>`)	The provider's `sub` maps to an `app_user`; unknown subjects are refused unless a sign-up flow links them
Find access	`user_profile`, `user_role`, `role`, `role_permission`	Exactly as for Colsynq tokens: profile in the resolved tenant, grants, assertions
Revoke	Provider + `app_user.status`	The provider revokes its tokens; Colsynq can still block the user or profile

The identity provider authenticates; Colsynq still decides society access. Profiles and roles never come from the provider.

26.9 Machine access with client credentials (later)

Step	Table or store	What happens
Partner gets a token	Identity provider	`client_credentials` grant with `private_key_jwt` or mTLS client authentication

Step	Table or store	What happens
Map client to Colsynq	`external_identity` (`provider = client:<issuer>`, `subject` = client ID)	Points to the service account's `app_user`
Find access	`user_profile` (the service account's only profile), `user_role`	Same gates and grants as API keys
Until then	`api_key`	Service accounts use tenant-bound API keys (section 26.3)

26.10 Responsibility summary

Concern	Owner	Tables	Not stored
Prove identity (password, MFA, passkey, provider)	IAM identity	`app_user`, `user_authenticator`, `backup_code`, `external_identity`	Plain passwords, provider tokens
Issue and renew sessions	IAM identity	`refresh_token`	Access tokens
Machine credentials	IAM identity	`api_key`	Plain keys
One-time links	IAM identity	`one_time_token`	Verification and join links (signed URLs)
Revoke	IAM identity	`app_user.token_version`, `refresh_token`, `api_key.revoked_at`	`jti` blocklist and reuse cache (Redis)
Society membership	IAM identity	`user_profile`	Tenant inside tokens
Access decisions	IAM access	`user_role`, `role`, `role_permission`, `permission`	Roles inside tokens
Evidence	IAM	`audit_event`	Personal data in payloads